Accessing Web APIs

Print to PDF

Contents

- 10.1. What is a Web API?
- 10.2. RESTful Requests
- 10.3. Accessing Web APIs
- 10.4. JSON Data
- 10.5. Resources
- 10.6. Acknowledgement

Most programming languages can load data from locally-saved `.csv` files. Moreover, they can also download data directly from websites on the internet. This allows scripts to always work with the latest data available, analyzing data that may be changing rapidly (such as from social networks or other live events). Web services may make their data easily accessible to computer programs by offering an **Application Programming Interface (API)**. A web service's API specifies *where* and *how* particular data may be accessed, and many web services follow a particular style known as *Representational State Transfer (REST)*. This chapter will cover how to access and work with data from these *RESTful APIs*.

10.1. What is a Web API?

An **interface** is the point at which two different systems meet and *communicate*: exchanging information and instructions. An **Application Programming Interface (API)** thus represents a way of communicating with a computer application by writing a computer program (a set of formal instructions understandable by a machine). APIs commonly take the form of **functions** that can be called to give instructions to programs — the set of functions provided by a library like `numpy` make up the API for that library.

While most APIs provide an interface for utilizing *functionality*, others provide an interface for accessing *data*. One of the most common sources of these data APIs is **Web**

services, which are websites that offer an interface for accessing their data or even services. For example, Alpha Vantage offers several financially related APIs to get stock quotes and news. OpenAI offers APIs to access the ChatGPT service.

With web services, the interface (the set of "functions" you can call to access the data) takes the form of **HTTP Requests** — a *request* for data sent following the ***H**yper**T**ext **T**ransfer **P**rotocol*. Your browser uses this same protocol (way of communicating) to access a web page! An HTTP Request represents a message that your computer sends to a web server (another computer on the internet which "serves", or provides, information). Upon receiving the request, that server determines what data to include in the **response** and sends that data *back* to the requesting computer. With a web browser, the response data takes the form of HTML files that the browser can *render* as web pages. With data APIs, the response data will be structured data you can process within your application.

In short, loading data from a Web API involves sending an **HTTP Request** to a server for a particular piece of data, receiving the **response**, parsing that response, and then processing it.

10.2. RESTful Requests

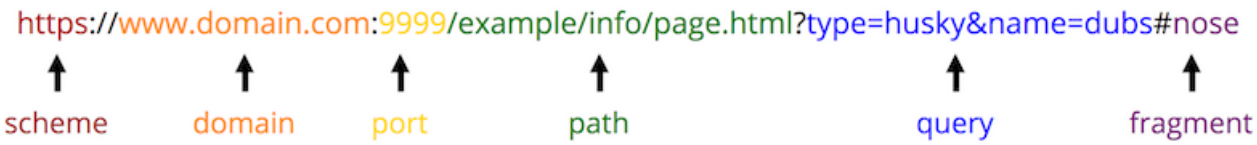
There are two parts to a request sent to an API: the name of the **resource** (data) you wish to access and a **verb** indicating what you want to do with that resource. In many ways, the *verb* is the function you want to call on the API, and the *resource* is an argument to that function.

10.2.1. URIs

Which **resource** you want to access is specified with a **Uniform Resource Identifier (URI)**. A URI is a generalization of a URL (Uniform Resource Locator) — what you commonly think of as "web addresses". URIs act a lot like the *address* on a postal letter sent within a large organization such as a university: you indicate the business address as well as the department and the person, and will get a different response (and different data) from Alice in Accounting than from Sally in Sales.

> **Note**
>
> The URI is the **identifier** (think: variable name) for the resource, while the **resource** is the actual *data* value you want to access.

Like postal letter addresses, URIs have a specific format to direct the request to the right resource.



Not all parts of the format are required — for example, you don't need a `port`, `query`, or `fragment`. Essential parts of the format include:

- `scheme` (`protocol`): the "language" the computer will use to communicate the request to this resource. With web services, this is normally `https` (**s**ecure HTTP)
- `domain`: the address of the web server to request information from
- `path`: which resource on that web server you wish to access. This may be the name of a file with an extension if you're trying to access a particular file, but with web services it often just looks like a folder path!
- `query`: extra **parameters** (arguments) about what resource to access.

The `domain` and `path` usually specify the resource. For example, `www.domain.com/users` might be an *identifier* for a resource which is a list of users. Note that web services can also have "subresources" by adding extra pieces to the path: `www.domain.com/users/mike` might refer to the specific "mike" user in that list.

With an API, the domain and path are often viewed as being broken up into two parts:

- The **Base URI** is the domain and part of the path that is included on *all* resources. It acts as the "root" for any particular resource. For example, the GitHub API has a base URI of `https://api.github.com/`.
- An **Endpoint**, or which resource on that domain you want to access. Each API will have *many* different endpoints.

  For example, GitHub includes endpoints such as:

- `/users/{user}` to get information about a specific `:user:` `id` (the `{}` indicate a "variable", in that you can put any username in there in place of the string `"{user}"`). Check out this example in your browser.
- `/orgs/{organization}/repos` to get the repositories that are on an organization page. See an example.

Thus, you can equivalently talk about accessing a particular **resource** and sending a request to a particular **endpoint**. The **endpoint** is appended to the end of the **Base URI**, so you could access a GitHub user by combining the **Base URI** (`https://api.github.com`) and **endpoint** (`/users/slankas`) into a single string: `https://api.github.com/users/slankas`. That URL will return a data structure of new releases, which you can request from your program or simply view in your web browser.

10.2.1.1. Query Parameters

To access only partial data sets from a resource (e.g., to only get some users), you also include a set of **query parameters**. These are like extra arguments that are given to the request function. Query parameters are listed after a question mark `?` in the URI and are formed as key-value pairs similar to how you named items in *lists*. The **key** (*parameter name*) is listed first, followed by an equal sign `=`, followed by the **value** (*parameter value*); note to include spaces and other special characters in the URIs, you have to percent encode those characters. For example, `%20` represents a space. You can include multiple query parameters by putting an ampersand `&` between each key-value pair:

```
?firstParam=firstValue&secondParam=secondValue&thirdParam=thirdValue
```

What parameter names you need to include (and what legal values to assign to that name) depends on the particular web service. Common examples include having parameters named `q` or `query` for searching, with a value being whatever term you want to search for: in `https://www.google.com/search?q=informatics`, the **resource** at the `/search` **endpoint** takes a query parameter `q` with the term you want to search for! You will need to view the service's documentation. Similarly, writing server-side endpoints requires you to provide that documentation to your users.

10.2.2. Access Tokens and API Keys

Many web services require you to register with them to send them requests. This allows them to limit access to the data and keep track of who is asking for what data (usually so that if someone starts "spamming" the service, they can be blocked). These keys also allow them to charge their users!

Many services provide **Access Tokens** (also called **API Keys**) to facilitate this tracking. These are unique strings of letters and numbers that identify a particular developer (like a secret password that only works for you). Web services will require you to include your *access token* as a query parameter in the request; the exact name of the parameter varies, but it often looks like `access_token` or `api_key`. When exploring a web service, watch for whether they require such tokens.

Access tokens act a lot like passwords; you will want to keep them secret and not share them with others. This means that you **should not include them in your committed files** so that the passwords don't get pushed to GitHub and shared with the world.

10.2.3. Managing Tokens and Keys

Several approaches exist to manage and store these "secrets":

10.2.3.1. Environment Variables

Your programs can read secret variables placed in environment variables. These variables be set in the shell: `export API_TOKEN=some value`. They can also be set in programs that call other programs (technically, this is the same as shell). This allows continuous integration pipelines to function. Docker can set these variables as well when starting containers — a common approach for configuring different environments for images. Cloud providers, such as AWS, provide configuration pages for environment variables for services such as Lambda.

Within Python, you'll use the `os` library to read these variables:

```
import os

api_token = os.getenv('API_TOKEN')
```

10.2.3.2. Configuration Files

API keys can be stored in configuration files, such as `.env`, `.json`, or `.yaml` files, to separate sensitive information from source code. This approach organizes API keys and allows developers to manage configuration settings without hardcoding them into the program. Tools like `python-dotenv` can load environment variables from `.env` files into the application, while libraries like json or PyYAML can parse other file formats. It's essential to restrict access to these files (i.e., only allowing a specific user to read the file) as well as to prevent these configuration files from being committed to a version control system (e.g., use .gitignore).

10.2.3.3. Key Management Systems

Key management systems (KMS) provide a secure and centralized way to store and manage API keys and other sensitive credentials. These systems, such as AWS Secrets Manager, HashiCorp Vault, or Azure Key Vault, encrypt stored keys and provide access control mechanisms to ensure that only authorized applications or users can retrieve them. API keys can be securely retrieved during runtime through SDKs, APIs, or environment variable injection, reducing the risk of hardcoding keys in source code. Many KMS solutions also support automated key rotation, versioning, and audit logging to track access and maintain compliance.

Store the configuration values in a `.env` file: (note the similarity to using environment variables)

```
API_KEY=your_api_ke_here
```

Then, load and use the API key:

```
import os
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

# Retrieve the API key
api_key = os.getenv('API_KEY')

# Use the API key
print(f"Using API Key: {api_key[:4]}...")  # Mask the key in logs for secu
```

10.2.4. Best Practices for Tokens and Keys

1. Place tokens in a secure location with limited permissions.
2. Limit the scope of tokens. Use tokens with the least privilege required for your use case and limit the token's lifetime.
3. Rotate tokens regularly to reduce the image of accidental exposure
4. Avoid hardcoding tokens. Do not directly include tokens in source code or print them in log files.
5. Handle tokens securely in memory by removing their value (setting to `null`) once they are no longer needed.

> **Warning**
>
> For client-side JavaScript, properly securing API keys is practically impossible. Assume local users can always view and modify these script files. In such situations, a server-side endpoint is often used as a proxy to access remote APIs.
>
> As an alternative, many advanced client pages will use OAuth to access remote resources. OAuth is a system for performing **authentification** — that is, letting someone log into a website from your application (like what a "Log in with Facebook" button does). OAuth systems require more than one access key, and these keys ***must*** be kept secret and usually require you to run a web server to utilize them correctly.

10.2.5. HTTP Verbs

When you send a request to a particular resource, you need to indicate what you want to *do* with that resource. When you load web content, you typically send a request to retrieve information (logically, this is a `GET` request). However, there are other actions you can perform to modify the data structure on the server. This is done by specifying an **HTTP Verb** in the request. The HTTP protocol supports the following verbs:

- `GET` Return a representation of the current state of the resource
- `POST` Add a new subresource (e.g., insert a record)
- `PUT` Update the resource to have a new state

- `PATCH` Update a portion of the resource's state
- `DELETE` Remove the resource
- `OPTIONS` Return the set of methods that can be performed on the resource

The most common verb is `GET`, used to "get" (download) data from a web service. Depending on how you connect to your API (i.e., which programming language you use), you'll specify the verb of interest to indicate what you want to do to a particular resource.

Overall, this structure of treating data on the web as a **resource** which we can interact with via **HTTP Requests** is referred to as the **REST Architecture** (REST stands for *REpresentational State Transfer*). This is a standard way of structuring computer applications that allows them to be interacted with in the same way as everyday websites. Thus, a web service that enables data access through named resources and responds to HTTP requests is known as a **RESTful** service with a *RESTful API*.

10.3. Accessing Web APIs

To access a Web API, you must send an HTTP Request to a particular URI. You can easily do this with the browser: navigate to a particular address (base URI + endpoint), and that will cause the browser to send a `GET` request and display the resulting data. For example, you can send a request to search GitHub for repositories named `d3` by visiting:

```
https://api.github.com/search/repositories?q=d3&sort=forks
```

This query accesses the `/search/repositories/` endpoint, specifying two query parameters:

- `q`: The term(s) you are searching for, and
- `sort`: The attribute of each repository that you would like to use to sort the results

(Note that the data you'll get back is structured in JSON format. See below for details).

In `Python` you can send GET requests using the `requests` library. As with other libraries not included with the Python distribution, you will need to install the package (library):

```
pip install requests
```

This library provides several functions that reflect HTTP verbs. For example, the `GET()` function will send an HTTP GET Request to the URI specified as an argument:

```
# Get search results for repositories named "d3"
import requests

url = "https://api.github.com/search/repositories?q=d3&sort=forks"
response = requests.get(url)

# Check the response status code
if response.status_code == 200:
    data = response.json()  # Parse JSON response
    print("First result:", data["items"][0])
else:
    print(f"Failed to retrieve data. HTTP Status Code: {response.status_coc
```

While it is possible to include *query parameters* in the URI string, `requests` also allows you to include them as a *dictionary*, making it easy to set and change variables (instead of formatting the string):

```
# Equivalent to the above, but easier to read and change
query_params = { "q": "d3", "sort": "forks" }
url = "https://api.github.com/search/repositories"
response = requests.get(url, query_params)
```

A server's response contains two parts: the **header**, and the **body**. You can think of the response as an envelope: the *header* contains metadata like the address and postage date, while the *body* contains the actual contents of the letter (the data).

In Python, `response.headers` is a specialized dictionary that contains the metadata.

```
import json

print(json.dumps(dict(response.headers),indent=4))
```

```
{
    "Date": "Thu, 02 Jan 2025 19:41:45 GMT",
    "Content-Type": "application/json; charset=utf-8",
    "Cache-Control": "no-cache",
    ...
}
```

As we are almost always interested in working with the *body*, we can extract that data from the response (e.g., open up the envelope and pull out the letter) using the "text" attribute of the response object.

```
body = response.text
```

More commonly, we'll use the data sent back in a particular format than just text. To access the response as a JSON object and convert to a Python dictionary, use:

```
answer = response.json()
```

> **Tip**
>
> Use `response.url` to check exactly what URI you sent the request to: copy that into your browser to make sure it goes where you expected!

10.4. JSON Data

Most APIs will return data in **JavaScript Object Notation (JSON)** format. Like CSV, this is a format for writing down structured data — but while `.csv` files organize data into rows and columns (like a data frame), JSON allows you to organize elements into **key-value pairs** similar to a Python *dictionary*! This allows the data to have a more complex structure, which is useful for web services.

In JSON, lists of key-value pairs (called *objects*) are put inside braces (`{ }`), with the key and value separated by a colon (`:`) and each pair separated by a comma (`,`). Key-value pairs are often written on separate lines for readability, but this isn't required. Note that keys need to be character strings (so in quotes), while values can either be character strings, numbers, booleans (written in lower-case as `true` and `false`), arrays, or even other objects! JSON also supports *arrays* of data.

For example:

```
{
  "first_name": "Ada",
  "job": "Programmer",
  "salary": 78000,
  "in_union": true,
  "languages" : [ "C++", "Javascript", "Python" ],
  "favorites": {
    "music": "jazz",
    "food": "pizza",
  }
}
```

(In JavaScript, the period `.` has special meaning, so it is not used in key names; hence the underscores `_`).

> **Note**
>
> Just as Python allows you to have nested dictionaries of dictionaries, JSON can have any form of nested *objects* and *arrays*.

JSON can also be structured as *arrays* of *objects*, such as a list of data about Seahawks games:

```
[
  { "opponent": "Dolphins", "sea_score": 12, "opp_score": 10 },
  { "opponent": "Rams", "sea_score": 3, "opp_score": 9 },
  { "opponent": "49ers", "sea_score": 37, "opp_score": 18 },
  { "opponent": "Jets", "sea_score": 27, "opp_score": 17 },
  { "opponent": "Falcons", "sea_score": 26, "opp_score": 24 }
]
```

The latter format is widespread in web API data: as long as each *object* in the *array* has the same set of keys, then you can easily consider this as a dataframe (aka, spreadsheet) where each *object* (keyed list) represents an **observation** (row), and each key represents a **feature** (column) of that observation.

With `pandas`, you can easily convert this into a data frame:

```
import pandas as pd

games = [
  { "opponent": "Dolphins", "sea_score": 12, "opp_score": 10 },
  { "opponent": "Rams", "sea_score": 3, "opp_score": 9 },
  { "opponent": "49ers", "sea_score": 37, "opp_score": 18 },
  { "opponent": "Jets", "sea_score": 27, "opp_score": 17 },
  { "opponent": "Falcons", "sea_score": 26, "opp_score": 24 }
]

df = pd.DataFrame(games)
print(df)
```

Output:

```
   opponent  sea_score  opp_score
0  Dolphins         12         10
1      Rams          3          9
2     49ers         37         18
3      Jets         27         17
4   Falcons         26         24
```

Tip

JSON data can be quite messy when viewed in your web browser. Installing a browser extension such as JSONView will format JSON responses in a more readable way, and even enable you to explore the data structure interactively.

10.5. Resources

- URIs (Wikipedia)
- HTTP Protocol Tutorial
- Programmable Web (list of web APIs; may be out of date)
- RESTful Architecture (original specification; not for beginners)
- JSON View Extension
- `requests` documentation

10.6. Acknowledgement

This chapter was from the INFO 201: Technical Foundations in Informatics book at info201/book under a under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License. Adapted for Python by John Slankas